Modern companies have moved away from that model. Today we hear terms like:

- Site reliability engineering (SRE)
- Production engineering
- Platform engineering
- Forward deployed engineering

The naming changes from company to company, but the overall objective is similar. These teams work closely with production systems. They monitor reliability, define service level objectives (SLOs), improve observability, and help reduce downtime.

In many companies, engineers are expected to participate in on-call rotations as well. This changes how software gets written. Developers start thinking more carefully about failure handling, retry logic, monitoring, alert quality, and recovery behaviour.

That shift is actually healthy for distributed systems.

The three pillars of observability

Before talking about advanced debugging techniques or AI tooling, we need to understand observability fundamentals. Most modern debugging workflows rely on three major pillars.

Metrics (Prometheus and Grafana): Metrics usually provide the first indication that something is wrong. Tools like Prometheus collect numerical system data over time, which includes request latency, CPU usage, memory utilisation, error rates, packet drops, queue sizes, and API throughput. Grafana helps visualise this data. A simple dashboard often reveals unexpected spikes or unusual patterns very quickly.

Metrics are extremely useful because they help answer: "When did the issue start?" But they usually do not explain the exact reason behind the failure.

Logs (Elasticsearch/OpenSearch): Logs provide much deeper operational details. When metrics indicate a problem, engineers usually start checking logs to understand which component failed, what error happened, which request triggered it, and whether the issue is repeating.

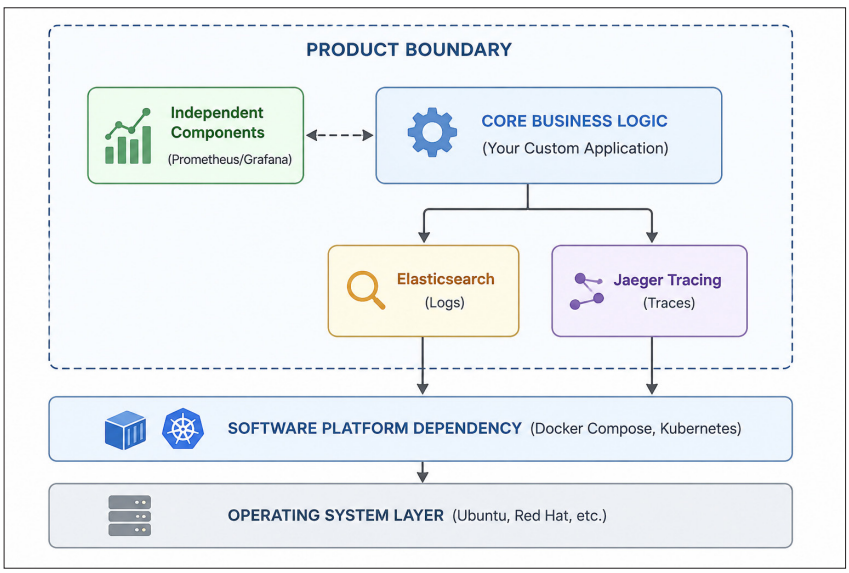


Figure 1: Observability architecture for modern distributed applications using metrics, logs, and tracing

Platforms like Elasticsearch or OpenSearch centralise logs from different services and make searching easier across distributed environments.

One important thing many teams learn late is that unstructured logs become painful very quickly. Structured logging with correlation IDs saves enormous debugging time in production systems.

Distributed tracing (Jaeger and OpenTelemetry): Tracing becomes critical once requests start crossing multiple services.

A single API request may travel through an API gateway, authentication service, caching layer, database service, messaging queue, analytics pipeline, and external APIs. Without tracing, engineers are mostly guessing where latency or failures are happening.

Distributed tracing tools such as Jaeger help visualise the complete request flow. Today many organisations are standardising OpenTelemetry for instrumentation because it provides a common framework for metrics, traces, and logs across different observability platforms.

Tracing becomes especially useful for intermittent latency issues, retry storms, service dependency failures, timeout analysis, and asynchronous message debugging.

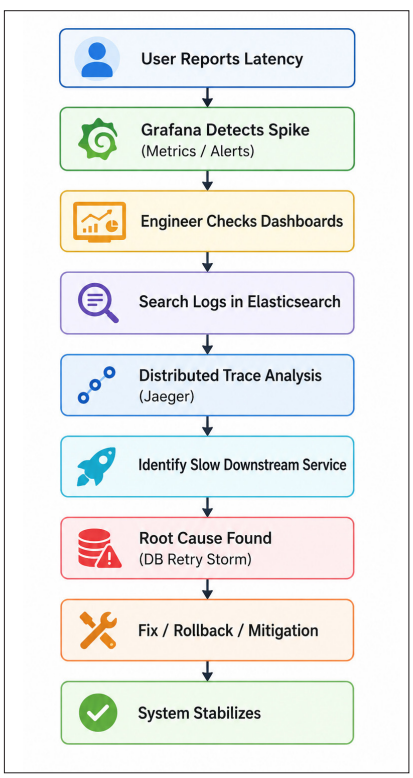


Figure 2: Typical debugging workflow in a distributed production system using metrics, logs, and distributed tracing